

This architecture follows a **Domain-Driven Design (DDD)** approach, focusing on security, role-based access control (RBAC), and high-performance financial data rendering.

1. UI Architecture

- **Framework:** Next.js 14+ (App Router) for Server-Side Rendering (SSR) to ensure security and SEO-friendly audit pages.
- **Language:** TypeScript (Strict mode).
- **Styling:** Tailwind CSS + Radix UI (Headless accessible components).
- **Communication:** TanStack Query (React Query) for server state; Axios with interceptors for secure token handling.
- **Data Visualization:** Recharts for AI financial insights and fraud trend analysis.

2. Folder Structure

```
src/
├── app/           # App Router (Pages & Layouts)
├── components/    # Shared UI (Button, Input, Modal)
├── features/      # Domain-specific logic (auth, banking, loans)
│   ├── auth/     # Login, MFA, Password recovery
│   ├── dashboard/ # Widgets, summary cards
│   ├── transactions/ # Transfer forms, history tables
│   └── loans/     # Application flow, status tracking
├── hooks/         # Global custom hooks (useUser, useAuth)
├── lib/           # API clients, axios config, theme
├── store/         # Zustand stores for client-side state
└── types/        # Global TypeScript interfaces
```

3. Core Components & UI Libraries

- **Components:**

TransactionTable*: High-performance virtualized list (TanStack Table).

SecurityBadge*: Status indicators for fraud/verified accounts.

MFAInput*: PIN-entry component with auto-focus movement.

LoanPipeline*: Kanban-style board for employee loan processing.

- **Libraries:**

* `lucide-react` (Iconography).

* `zod` (Form schema validation).

* `react-hook-form` (Performant form handling).

* `date-fns` (Date manipulation for statements/audits).

4. Routing & Authentication

- **Middleware:** Check JWT validity on every route transition.

- **Guards:**

* `isLoggedIn`: Redirect to login if not authenticated.

* `hasRole`: Role-based route access (Customer vs. Employee vs. Admin).

- **Flow:**

1. Login Form → 2. MFA Challenge → 3. Role-based Dashboard.

5. State Management

- **Server State:** TanStack Query for all API interactions (caching, loading states, error handling).
- **Client State:** Zustand for global UI state (e.g., sidebar collapse state, current active account context, theme).

6. Forms & Validation

- **Approach:** All forms will use `zod` schema-first validation.
- **Example (Transfer):**

```typescript

```
const transferSchema = z.object({
```

```
 amount: z.number().min(0.01).max(10000), // Enforce business rules
```

```
 recipientAccount: z.string().length(12),
```

```
mfaCode: z.string().length(6)
```

```
});
```

```
...
```

---

## 7. Dashboard Layout

- **Layout Pattern:** Shell-based structure.
- \* **Sidebar:** Persistent navigation with role-specific items.
- \* **Top Bar:** Global search (transactions, accounts), Notifications (Fraud alerts), and User Profile.
- \* **Main Content Area:** Dynamic grid/flex layout for widgets (Balance cards, recent activity, AI insight tips).

---

## 8. Security Considerations (Architecture Level)

- **Request Interceptors:** Automatic inclusion of `Authorization: Bearer `.
- **Response Interceptors:** Global handling of 401 (unauthorized) and 403 (forbidden) status codes to trigger logout or redirect.
- **Data Masking:** HOC (Higher-Order Component) or utility to mask account numbers in the UI (e.g., `\*\*\*\*1234`).
- **CSRF Protection:** Secure cookies for storing session tokens.

---

## 9. Development Roadmap

1. **Phase 1:** Auth System & MFA integration.
2. **Phase 2:** Account Overview & Transaction history.
3. **Phase 3:** Loan application & Review interface.
4. **Phase 4:** AI Insights & Dashboard metrics (Admin view).

This architecture ensures that the **Banking Management System** remains modular, type-safe, and highly responsive, which is critical for financial applications where data integrity and accessibility are paramount.